

NOIP 模拟赛

题目名称	棋盘翻转	农场大户	朋友圈圈	简单序列
题目类型	传统型	传统型	传统型	传统型
目录	board	farm	friend	sequence
提交源程序文件名	board.cpp	farm.cpp	friend.cpp	sequence.cpp
可执行文件名	board	farm	friend	sequence
输入文件名	board.in	farm.in	friend.in	sequence.in
输出文件名	board.out	farm.out	friend.out	sequence.out
每个测试点时限	2.5 秒	1.5 秒	1.0 秒	4 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	25	25	25	20
测试点是否等分	是	是	是	是
比较方式	全文比较	Special Judge	全文比较	全文比较

编译选项: `-O2 -std=c++14 -static`。

棋盘翻转

题目描述

一个 $n \times n$ 的棋盘上放有 n 个车，满足棋子之间不会互相攻击，即每行每列只有一个车。而 wshcl 认为整齐的排列是好看的，即第 i 列的车应该处于第 i 行。

定义一次操作为：选择一个列的区间 $[l, r]$ ，并将这些列沿 $[l, r]$ 的中轴左右翻转，代价为区间长度 $r - l + 1$ 。

定义操作的总代价为每次操作代价的按位异或和，需要用若干次操作满足 wshcl 要求。

求所有可行方案中总代价的最小值和最大值。

输入格式

从文件 `board.in` 中读入数据。

第一行一个整数 T 表示数据组数。接下来依次描述各组数据，对于每组数据：

第一行一个正整数 n ，表示棋盘的边长及车的数量。

接下来 n 行，每行 2 个整数，分别表示每个车的坐标。

输出格式

输出到文件 `board.out` 中。

对于每组数据，输出一行两个整数，分别表示总代价的最小值和最大值。

样例 #1

样例输入 #1

1	1
2	6
3	6 4
4	5 3
5	1 1
6	3 5
7	2 2
8	4 6

样例输出 #1

1	0 5
---	-----

样例解释 #1

最小价值：翻转 $[4, 5]$ 、 $[3, 4]$ 、 $[5, 6]$ 、 $[4, 5]$ 即可，价值为 $2 \oplus 2 \oplus 2 \oplus 2 = 0$ 。

最大价值：翻转 $[3, 6]$ 、 $[3, 4]$ 、 $[5, 6]$ 、 $[1, 1]$ 即可，价值为 $4 \oplus 2 \oplus 2 \oplus 1 = 5$ 。

样例 #2

见附加文件中的 `farm2.in` 和 `farm2.ans`。

数据范围

对于所有数据：

- $T \leq 5$ ；
- $n \leq 5 \times 10^5$ ；
- 保证给出的坐标合法。

测试点编号	$n \leq$
1 ~ 5	10
6 ~ 10	100
11 ~ 15	10^3
16 ~ 20	10^5
20 ~ 25	5×10^5

农场大户

题目描述

有一个大农场，各个区域组成一个 $n \times n$ 的矩阵，其中第 i 行第 j 列的区域收益为 $a_{i,j}$ 。

wshcl 想买下一部分区域，而为了便于管理，这片区域应当形如一块子矩形。定义子矩形 $(x_1, x_2, y_1, y_2) (x_1 \leq x_2, y_1 \leq y_2)$ 为以 (x_1, y_1) 为左上角， (x_2, y_2) 为右下角组成的矩形区域。

买下某片区域后需要用围栏将该区域围起来，定义子矩形 (x_1, x_2, y_1, y_2) 的周长为：

$$C(x_1, x_2, y_1, y_2) = 2 \times (x_2 - x_1 + 1) + 2 \times (y_2 - y_1 + 1)$$

围住该子矩形所需的围栏数量即为周长。

定义子矩形 (x_1, x_2, y_1, y_2) 的总收益为：

$$S(x_1, x_2, y_1, y_2) = \sum_{i=x_1}^{x_2} \sum_{j=y_1}^{y_2} a_{i,j}$$

wshcl 追求性价比，因此你需要帮他求出总收益和周长的最大比值，即：

$$\max_{1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq n} \frac{S(x_1, x_2, y_1, y_2)}{C(x_1, x_2, y_1, y_2)}$$

在此基础上，你还需给出一个方案，这样 wshcl 才能理解你的想法并做出最优的选择。

输入格式

从文件 `farm.in` 中读入数据。

第一行输入一个正整数 n ，表示矩阵的边长。

接下来 n 行，每行 n 个整数，其中第 i 行第 j 列的数字表示 $a_{i,j}$ 。

输出格式

输出到文件 `farm.out` 中。

对于每组数据，第一行一个浮点数，表示总收益和周长的最大比值。

第二行输出两个空格隔开的整数 y_1, x_1 ，第三行输出两个空格隔开的整数 y_2, x_2 ，表示其中一个方案为 (x_1, x_2, y_1, y_2) 。若有多种方案符合条件，任意输出一即可。

你的答案被认为正确，当且仅当相对误差或绝对误差不超过 10^{-7} 。你的子矩形被认为正确，当且仅当其 $\frac{S}{C}$ 值与你的输出绝对误差或相对误差不超过 10^{-7} 。

形式化地，设你的答案为 a ，正确答案为 b ，允许的误差范围为 $\epsilon = 10^{-7}$ ，则你的答案被判为正确，当且仅当 $\min\{|\frac{a-b}{b}|, |a-b|\} \leq \epsilon$ 。

样例 #1

样例输入 #1

```
1 2
2 100 100
3 1 1
```

样例输出 #1

```
1 33.3333333333
2 1 1
3 2 1
```

样例 #2

见附加文件中的 `farm2.in` 和 `farm2.ans`。

样例 #3

见附加文件中的 `farm3.in` 和 `farm3.ans`。

数据范围

对于所有数据：

- $1 \leq n \leq 500$;
- $|a_{i,j}| \leq 10^9$ 。

每个测试点的具体限制如下表所示：

测试点编号	$n \leq$
1 ~ 5	100
6 ~ 10	200
11 ~ 15	300
16 ~ 20	400
21 ~ 25	500

朋友圈圈

题目描述

广场上有 n 个人，编号为 $1 \sim n$ ，第 i 个人的权值为 a_i 。他们之间存在特殊的朋友关系：

- 对于两个人 x, y ，若 $\gcd(a_x, a_y)$ 为合数，则 x, y 为直接朋友。
- 对于三个人 x, y, z ，若 x, y 为朋友， y, z 为朋友，则 x, z 为间接朋友。

wshcl 需要选出一个人的集合作为朋友圈，满足集合内任意两个人均为直接或间接的朋友。

在此之前，wshcl 必须强制移去一个人，注意移去一个人之后可能会破坏某些间接朋友的关系。

对于所有 n 种移去人的方案，求 wshcl 能选出的最大集合大小的最小值。

输入格式

从文件 `friend.in` 中读入数据。

第一行一个整数 T 表示数据组数。接下来依次描述各组数据，对于每组数据：

第一行一个正整数 n ，表示总人数。

第二行 n 个正整数 $a_1 \sim a_n$ ，表示每个人的权值。

输出格式

输出到文件 `farm.out` 中。

对于每组数据，输出一行一个整数，表示最大集合大小。

样例 #1

样例输入 #1

```
1 3
2 5
3 8 4 12 18 9
4 5
5 36 20 84 45 231
6 7
7 100 200 300 400 500 600 700
```

样例输出 #1

```
1 2
2 3
3 6
```

样例 #2

见附加文件中的 `friend2.in` 和 `friend2.ans`。

数据范围

对于所有数据：

- $1 \leq T \leq 10$ ；
- $2 \leq n \leq 10^5$ ；
- $2 \leq a_i \leq 10^7$ 。

每个测试点的具体限制如下表所示：

测试点编号	$n \leq$	$a_i \leq$
1 ~ 2	300	2000
3 ~ 4	300	10^7
5 ~ 7	5000	30000
8 ~ 10	5000	10^7
11 ~ 18	10^5	10^5
19 ~ 25	10^5	10^7

简单序列

题目描述

wshcl 有一个长度为 n 的正整数序列 $a_{1\sim n}$ ，并且一开始就给出一个常数 k ，求：

$$\max_{1 \leq i \leq j \leq n, |i-j| \leq k} \{a_i + a_j\}$$

但是 wshcl 觉得这根本难不倒你，于是 wshcl 给出了 q 次修改操作，形如将 a_x 修改为 y 。

除了第一次修改前你需要给出答案之外，每次修改后你也都需要求出答案。

特别的，对于一部分测试点，我们有方法让你强制在线。

输入格式

从文件 `sequence.in` 中读入数据。

输入的第一行包括四个正整数 n, k, q, op ，分别表示序列长度、给定常数、操作次数、是否强制在线。

接下来一行 n 个数，表示一开始的序列 $a_{1\sim n}$ 。

接下来 q 行，每行包括两个整数 x 和 y ，表示将第 x 个数修改为 y 。当 $op = 1$ 时，你需要将输入中所有的数异或上上一次输出的结果。

输出格式

输出到文件 `sequence.out` 中。

输出 $q + 1$ 个数，表示询问的答案。

样例 #1

样例输入 #1

```
1 4 2 1 0
2 6 1 2 4
3 1 3
```

样例输出 #1

```
1 8
2 6
```

样例 #2

样例输入 #2

```
1 4 2 1 1
2 6 1 2 4
3 9 11
```

样例输出 #2

```
1 | 8
2 | 6
```

样例 #3

见附加文件中的 `sequence3.in` 和 `sequence3.ans` , 该样例满足测试点 2 ~ 4 的限制。

样例 #4

见附加文件中的 `sequence4.in` 和 `sequence4.ans` , 该样例满足测试点 9 ~ 11 的限制。

样例 #5

见附加文件中的 `sequence5.in` 和 `sequence5.ans` , 该样例满足测试点 19 ~ 20 的限制。

数据范围

对于所有数据：

- $1 \leq k < n, q \leq 10^6$;
- $1 \leq x' \leq n$;
- $1 \leq y' \leq 10^9$ 。

其中 x', y' 为每次操作的实际值。

测试点编号	$n \leq$	$q \leq$	$op =$
1	5000	5000	1
2 ~ 4	10^5	10^5	0
5 ~ 8	10^6	10^5	0
9 ~ 11	10^5	10^5	1
12 ~ 14	10^6	10^5	1
15 ~ 18	3×10^5	3×10^5	1
19 ~ 20	10^6	10^6	1